

Improving the capabilities of static analysis for bug detection using LLMs

Sohaib Ahmad
FAST-NU, Lahore, Pakistan
l215635@lhr.nu.edu.pk

Muhammad Hassan Khalid
FAST-NU, Lahore, Pakistan
l215692@lhr.nu.edu.pk

Muhammad Hamza
FAST-NU, Lahore, Pakistan
l215636@lhr.nu.edu.pk

ABSTRACT

Static analysis is a crucial method that is used in the software engineering industry for the identification of bugs, vulnerabilities and issues related to code quality. This term paper talks about the traditional ways static analysis is being carried out, the limitations and how the integration of Large Language Models (LLMs) can help the industry in carrying out static analysis in a better way by overcoming the potential problems in the industry. The research states the capabilities of the LLMs such as their ability to provide contextual understanding, reduce the number of false positives and by enhancing the accuracy of static analysis tools. This paper also states the implementation of LLMs, the current research advancements in the field and prospects for the development of static analysis tools. The paper aims to provide a comprehensive overview of static analysis tools that can improve software reliability and security.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation**; *Software testing and debugging*; • **Computing methodologies** → **Natural language processing**; • **General and reference** → *Surveys and overviews*;

KEYWORDS

Static Analysis, Large Language Models, Bug Detection, Software Security, Code Analysis, AI in Software Engineering, Vulnerability Detection, Contextual Analysis, LLM-based Tools, Software Reliability

1 INTRODUCTION

1.1 Overview of Static Analysis

Static Analysis is a technique of examining and evaluating software code without running it. The primary aim is to identify potential errors, vulnerabilities and compliance issues early in the development cycle. This process is crucial in ensuring software quality as it allows the developers to detect and rectify issues at an early stage, thus, reducing the cost and complexity of the fixes needed.

Static Analysis consists of several approaches and tools that inspect the source code to verify that the code meets the coding standards, identify bugs and highlight vulnerabilities in the source code. Unlike dynamic analysis, which occurs by the execution of the code, static analysis examines the code structure, syntax and semantics to predict how the code will behave. By doing so, developers make sure that their code follows best practices and meets the requirements for maintainability, security and performance.

Common static analysis tools such as Coverity, SonarQube and CodeQL automate the process of examining the code to detect

common patterns that may lead to errors. These tools can identify a range of issues from simple syntax errors to more complex logical flows and provide developers with actionable insights to make their code better and safer. Static analysis is particularly valuable in large projects, where manual code reviews become impractical due to the large size and amounts of the code, also, in safety critical systems where the cost of failure is high.

1.2 Current Static Analysis Methods

Static Analysis consists of several traditional methods that do not use the Large Language Models (LLMs). These include:

- **Data Flow Analysis:** The flow of data is tracked throughout the program to identify anomalies such as uninitialized variables, variable misuse or potential memory leaks. It is very important to understand how values propagate throughout the program.
- **Control Flow Analysis:** The flow of control is examined in this analysis within the program's structure to make sure that all potential execution paths are managed correctly. This helps in determining issues such as dead code or unreachable sections of code.
- **Symbolic Execution:** This involves analyzing code using symbolic rather than concrete inputs to determine conditions under which specific code paths are executed. It is helpful in the identification of input-related errors and potential vulnerabilities.
- **Abstract Interpretation:** In this approach, the behavior of a program is approximated by analyzing it with an abstract model. It helps in verifying properties like variable types and the detection of potential runtime code errors without having to actually execute the code.
- **Linting:** In this rule-based checks are applied to identify stylistic issues, coding standard violations and common errors which include incorrect indentation or undeclared variables. Tools such as PyLint for Python or ESLint for JavaScript are used for the linting.
- **Pattern Matching:** This method uses pre-defined patterns to detect the potential issues or security issues in the code. This approach is very effective for detecting insecure functions or commonly known vulnerability patterns.
- **Type Checking:** This approach is used in statically typed languages to validate that variables and functions are used in ways that conform to declared data types. It helps to prevent type-related errors before the execution of the code.

These traditional methods help to provide a foundation for identifying issues in codebases even though they are limited due to their

focus on the on surface-level code structure and having difficulty in understanding deeper meanings or logic.

2 RESEARCH METHODOLOGY

The research being stated in this paper is based on a thorough review of existing literature and tools in the field of static analysis and large language models. Academic journals, conference proceedings and technical reports from established research institutions have been used to compile this paper's research. The methodology involves comparing various static analysis approaches both traditional and LLM-based to understand the strengths and weaknesses of each. We also identified the limitations and the areas for potential improvement for both approaches. Moreover, the analysis consists of a qualitative assessment of current static analysis tools and the integration of LLMs extracting insights from recent studies and experimental results reported in the literature.

2.1 Challenges in Static Analysis

Even though static analysis has a load of benefits, it still faces considerable challenges. One major problem is the false positives, that is the instances where the tool flags non-issues as errors. These inaccuracies burden developers with unnecessary alerts which thus complicate the debugging process and hence reduce the trust in these tools. As highlighted in "Static Analysis Techniques and Tools: A Systematic Mapping Study" [4] false positives remain a problem due to which developers often overlook genuine issues due to the high frequency of false alarms.

Static analysis has also limitations in contextual understanding particularly when analyzing languages with dynamic features such as Python and JavaScript. Traditional static analyzers primarily rely on syntactic patterns which often do not have the depth required to grasp the deeper meaning and subtle details in the code. Also, scalability is a major problem as static analysis tools may become less effective or slow down as software systems increase in size and complexity, as stated in the Google's research on the scalability of static analysis tools "Lessons from Building Static Analysis Tools at Google" [5].

3 THE ROLE OF LARGE LANGUAGE MODELS (LLMs) IN STATIC ANALYSIS

3.1 Introduction to LLMs

Large Language Models (LLMs) such as OpenAI's GPT and Google's BERT have shown exceptional capabilities in text understanding and generation. They are powered by extensive datasets and deep learning algorithms. LLMs can parse and interpret unstructured data, making them particularly effective in recognizing patterns, syntax and semantic meaning in the code. LLMs extensive training help them process programming languages and detect complex relationships within codebases.

LLMs represent a promising development in enhancing static analysis capabilities which is discussed in "The Emergence of Large Language Models in Static Analysis" [1]. LLMs can provide a deeper understanding of how code is written and what it means thus overcoming the weaknesses of traditional static analysis tools such

as struggling to understand the full context of the code and flagging issues that are not actually problems.

3.2 Potential of LLMs in Static Analysis

LLMs enhance static analysis by using a deeper understanding of meaning of the code and making it easier to detect issues more accurately. The traditional tools rely on syntactic patterns which can result in missed context-specific bugs or the flagging of non-issues as errors. On the other hand, LLMs can interpret the intent behind the code constructs enabling more precise identification of genuine issues. In "Assisting Static Analysis with Large Language Models: A ChatGPT Experiment" [2] it is illustrated how LLMs can generate detailed code summaries and significantly reduce false positives allowing developers to focus on actual issues.

LLMs perform very well in tasks requiring contextual understanding. They can detect subtle errors, such as incorrect API usage or uninitialized variables, which may be overlooked by traditional tools. The research "Can Large Language Models Replace Static Analysis Tools?" [3] supports the supplementary role of LLMs in making the traditional analysis tools better by identifying complex and context-sensitive issues.

3.3 Examples of Use Cases

- Security Vulnerability Detection: LLMs can identify patterns indicating security vulnerabilities such as SQL injection or cross-site scripting risks, as outlined in "Static Analysis Techniques and Tools: A Systematic Mapping Study" [4].
- Improved Variable Usage Detection: LLMs can detect errors such as uninitialized variables and improper variable handling which are often missed by traditional static analyzers.

4 CURRENT RESEARCH AND TOOLS

4.1 Existing Tools and Techniques

There are well established tools in the field of static analysis such as Coverity, CodeQL and SonarQube each meeting different development needs. Even though these tools are instrumental in enforcing coding standards they are limited by high false-positive rates and inadequate contextual insight. In "Comprehensive Evaluation of Static Analysis Tools for Their Performance in Finding Vulnerabilities in Java Code" [6] a study analyzing different types of static analysis tools, highlighted the strengths and weaknesses of different tools. Moreover, the need for more advanced methods that can minimize unnecessary alerts is emphasized in the study.

4.2 Research Advances

Current research is being conducted in exploring the integration of LLMs with static analysis frameworks. For instance, "The Emergence of Large Language Models in Static Analysis" [1] found that LLMs improved call graph analysis accuracy by 30 percent in certain tasks while "Reducing False Positives in Vulnerability Detection with Language Models" [1] stated a 40 percent reduction in false positives when LLMs were integrated in the static analysis tools. These findings enlighten the potential of LLMs to enhance traditional tools by offering deeper and contextually aware insights.

5 IMPLEMENTING LLM-ENHANCED STATIC ANALYSIS

5.1 Framework for Integration

The integration of LLMs into the static analysis process is completed in a number of stages:

- **Data Preprocessing:** The code data goes through the processes of tokenization and normalization. The data must also be rendered fit for LLM application to perform effective analysis.
- **Model Training:** LLMs are typically fine-tuned on domain-specific code datasets, allowing specialization in specific bug detection tasks or programming languages.
- **Deployment:** After the completion of training, they put the LLM within the static analysis with the help of which the LLMs are able to provide information on how the code is being used while the coding is going on.

As stated in "Static Analysis: A Survey of Techniques and Tools," [3] this structure serves as a starting point for comprehending the extent of LLM-enhanced static analysis improvement.

5.2 Challenges in Implementation

- **Data Privacy:** Data by LLMs usually includes source code and in most cases, there are ethical implications of such an inclusion and processing; in other words, customary software development code is mostly protected by intellectual property rights. Some of these problems can be alleviated by, for example, ensuring high levels of code anonymization and using installation of the tools within an organization's locations as it has been appreciated in "Lessons from Building Static Analysis Tools at Google". [5]
- **Model Complexity:** Some LLMs are also very demanding on resources and training them is very expensive. Some methods like model pruning and distillation have been developed to ensure that these high performing models are not too costly to be used.

5.3 Possible Solutions

Specializing in smaller models for targeted tasks has proven effective in reducing false positives while maintaining high accuracy. Targeted fine-tuning, as discussed in "The Emergence of Large Language Models in Static Analysis," [1] is also a viable option when the challenges of computational accuracy versus cost is put into perspective.

6 CASE STUDY AND HYPOTHETICAL SCENARIOS

Consider the following better Python function:

```
def foo():
    x = 10
    if x > 5:
        y = x * 2
    else:
        y = x + 2
    print(y) # Using 'y' after proper initialization and c
            ontext-aware assignment
```

In this example, a traditional static analyzer might flag irrelevant issues, such as naming conventions, while failing to identify the critical error of using a variable before it is initialized. On the other hand, an LLM based tool can accurately identify the "use-before-initialization" error by understanding the code flow and context. LLMs offer context-sensitive insights thereby effectively reducing unnecessary alerts as shown in "Assisting Static Analysis with Large Language Model" [6].

7 BENEFITS AND LIMITATIONS

7.1 Advantages

- **Enhanced Bug Detection Accuracy:** LLMs contextual understanding helps in reducing false positives and improving detection accuracy.
- **Scalability for Complex Codebases:** Fine-tuned LLMs scale more effectively with large codebases compared to traditional tools.
- **Detailed Explanations:** LLMs help in providing meaningful explanations for flagged issues thus fostering a deeper understanding of code errors.

7.2 Drawbacks

- **High Computational Costs:** Running large LLMs is resource-intensive process hence a need for necessitating optimization strategies.
- **Training Data Quality Dependency:** The effectiveness of LLMs depends on high-quality and unbiased training data as stated in "Efficacy of Static Analysis Tools for Software Defect Detection." [6]
- **Potential Biases:** LLMs might unintentionally develop biases which may impact the accuracy of bug detection.

8 FUTURE PROSPECTS

8.1 Emerging Trends

The creation of self-improving static analysis tools that take into use ongoing feedback from LLMs is approaching. "Large Language Models in Source Code Static Analysis" [1] highlights the potential of feedback loops to enable ongoing learning, making static analysis tools increasingly intelligent over time.

8.2 Long-Term Potential

As LLMs advance, they could enable fully automated code review systems capable of proactively identifying vulnerabilities. This evolution could fundamentally transform software development by shifting the focus from reactive debugging to proactive vulnerability mitigation.

Specific Research Questions and Open Challenges: There remain several research questions to be addressed to fully realize the potential of LLMs in static analysis. These include investigating the limits of LLMs in very largescale codebases, improving latency performance on LLM models for more timely analysis, and finding ways on how to mitigate LLM-based prediction biases. Exploring different hybrid or alternatively branded architectures may also be one of the areas of focus in the future to tackle the more advanced issues that the current state-of-the-art faces; these would incorporate elements of LLMs as well as some level of symbolic reasoning.

9 CONCLUSION

The integration of Large Language Models in static analysis can help create a significant advancement in software engineering. By

reducing false positives and providing contextually enriched insights, LLMs enhance the reliability and efficacy of static analysis tools, supporting the development of more secure and robust software systems.

REFERENCES

- [1] Ashwin Prasad Shivarpatna Venkatesh et al. The Emergence of Large Language Models in Static Analysis: A First Look through Micro-Benchmarks. ACM, 2023.
- [2] Haonan Li et al. Assisting Static Analysis with Large Language Models: A ChatGPT Experiment. ACM, 2023.
- [3] Han Cui. Can Large Language Models Replace Static Analysis Tools? SPIE, 2023.
- [4] Static Analysis: A Survey of Techniques and Tools. Springer.
- [5] Lessons from Building Static Analysis Tools at Google. Google Research.
- [6] Efficacy of Static Analysis Tools for Software Defect Detection. Papers With Code.